
Sparseformer: Improving Long-Range Dependency Modeling for Time-Series Forecasting

Haiyang Gao
Wuhan University

Working Draft - July 14, 2026

Abstract

Time-series forecasting is central to financial decision-making, yet financial data are noisy, non-stationary, heavy-tailed, and difficult to model with conventional parametric methods. Recent Transformer-based architectures improve long-range dependency modeling through attention mechanisms, but most retain standard feed-forward neural networks as the nonlinear transformation module. This paper investigates whether replacing this component with Kolmogorov–Arnold Networks (KANs) can improve portfolio-oriented financial forecasting. We develop a Transformer–KANs forecasting framework that combines endogenous stock-return patches with exogenous market information through self-attention and cross-attention, and then maps the learned representation directly into portfolio weights. Instead of optimizing a conventional prediction loss such as mean squared error, the model is trained under a negative Sharpe-ratio objective, aligning the learning problem with the risk-adjusted performance criterion used in portfolio construction. We further employ Selective Learning to reduce the influence of noisy, anomalous, and less generalizable financial observations. Empirically, the proposed model achieves an out-of-sample Sharpe ratio of 1.375. These results suggest that KAN-enhanced Transformer architectures offer a promising direction for integrating long-range temporal modeling, nonlinear function approximation, and Sharpe-ratio-driven portfolio allocation.

1 Introduction

Financial time series forecasting has long been a central problem in empirical finance, as accurate forecasts of asset returns, volatility, and risk dynamics are directly related to portfolio allocation, risk management, derivative pricing, and trading strategy design. Unlike many physical time series, financial data are typically noisy, non-stationary, heavy-tailed, and characterized by volatility clustering, making prediction particularly challenging. Classical statistical models laid the foundation for this field. For example, autoregressive moving average (ARMA) models provide a systematic framework for modeling linear temporal dependence [Box and Jenkins, 1976], while the autoregressive conditional heteroskedasticity (ARCH) model of [Engle, 1982] and its generalized extension, the GARCH model of [Bollerslev, 1986], became standard tools for capturing time-varying volatility in financial markets. These models have been widely used because they offer interpretable parametric descriptions of conditional mean and conditional variance dynamics. However, their reliance on relatively restrictive assumptions and limited capacity to capture complex nonlinear dependencies motivates the development of more flexible forecasting models, especially in high-dimensional stock markets where cross-sectional interactions and long-range temporal patterns are essential for portfolio decision-making.

In recent years, the rapid development of deep learning has shifted financial time series forecasting from purely parametric models toward highly expressive neural architectures. Among these methods, Transformer-based models have become one of the dominant research directions for long-sequence time series forecasting, largely because their self-attention mechanism is well suited to modeling long-range temporal dependencies. Starting from the original Transformer architecture [Vaswani et al., 2023], a series of forecasting-specific variants have been proposed to address the computational and statistical challenges of long sequences. Informer improves efficiency through ProbSparse self-attention [Zhou et al., 2021]; Autoformer incorporates decomposition and auto-correlation mechanisms to better capture trend and seasonal structures [Wu et al., 2022]; FEDformer further combines seasonal-trend decomposition with frequency-domain representations [Zhou et al., 2022]; and Non-stationary Transformer explicitly tackles distribution shifts in real-world non-stationary series [Liu et al., 2022]. More recent models continue this line of development: PatchTST uses subseries-level patching and channel independence to improve long-term forecasting [Nie et al., 2023]; iTransformer reconsiders the role of attention by applying it across variate tokens rather than temporal tokens [Liu et al., 2024]; Fredformer addresses frequency bias in Transformer forecasting [Piao et al., 2024]; DeformableTST reduces the over-reliance on patching through deformable sparse attention [Luo and Wang, 2024]; and TimeXer extends Transformer forecasting to settings with exogenous variables by jointly modeling endogenous temporal patches and exogenous variate-level information [Wang et al., 2024]. This trend is especially relevant for financial forecasting, where asset returns are influenced by long-memory effects, cross-asset interactions, macroeconomic conditions, and abrupt regime changes.

The effectiveness of Transformer-based models mainly comes from two architectural ingredients. First, the attention block learns data-dependent weights over different time steps or variables, allowing the model to selectively focus on historically relevant observations and thereby capture long-range temporal dependencies that are difficult for recurrent or local models to preserve. Second, the attention block is followed by a deep feed-forward neural network, which applies nonlinear transformations to the learned representations. This component increases the expressive power of the model and enables it to capture complex nonlinear relationships among market signals, volatility patterns, and asset-level interactions. These properties make Transformer-based architectures a natural candidate for portfolio-oriented forecasting tasks, where the objective is not only to predict future price movements but also to extract predictive representations that support allocation decisions and risk-adjusted performance optimization.

While most Transformer based forecasters retain the standard feed-forward neural network as the nonlinear transformation module after attention, this design choice is rarely examined in isolation, especially in financial applications. An emerging alternative is the Kolmogorov–Arnold Network (KANs), which is motivated by the Kolmogorov–Arnold representation theorem and replaces scalar weights or fixed node-wise activations with learnable univariate functions, commonly parameterized by splines [Liu et al., 2025]. This structure is attractive because it preserves the ability to model nonlinear relationships while potentially using a more compact and interpretable functional representation than conventional MLPs. Recent studies have begun to explore KANs in long-term time series forecasting, suggesting that adaptive basis functions can serve as a flexible modeling core for capturing complex temporal patterns [Jiang et al., 2025]. Nevertheless, their role in financial forecasting remains largely underexplored, with only preliminary evidence on standard KANs for stock price prediction and limited understanding of how KANs-based nonlinear modules interact with attention-based architectures [Rather et al., 2025]. Therefore, in this paper, we replace the neural network component in the Transformer feed-forward network with KANs, and investigate its effect on portfolio-oriented prediction and Sharpe-ratio-driven allocation in financial time series forecasting.

This leads to the main contribution of our study. We develop a portfolio-oriented Transformer–KANs model that directly maps predictive information into portfolio weights. Specifically, the model takes as input a panel of historical stock returns, Fama–French three-factor variables, and a market news sentiment signal, with a total input dimension of 34. The output is a vector of portfolio weights optimized toward maximizing the Sharpe ratio, a standard risk-adjusted performance criterion in portfolio construction [Sharpe, 1966]. This formulation is important because, in investment applications, accurate return prediction is not sufficient by itself: the ultimate objective is to translate predictive signals into allocations that deliver superior compensation per unit of risk.

Our model has several distinctive features. First, we fuse Transformer-based temporal modeling with Kolmogorov–Arnold Networks by replacing the neural network component in the Transformer

feed-forward network with KANs-style nonlinear transformations. To the best of our knowledge, this design remains underexplored in financial time series forecasting and provides a direct way to examine whether learnable spline-based nonlinearities can improve portfolio prediction. Second, in addition to historical returns, the model incorporates cross-sectional and market-level information, including the Fama–French three factors [Fama and French, 1993] and market news sentiment, allowing it to exploit both time-series dynamics and broader systematic signals. Third, we train the model using the Adam optimizer [Kingma and Ba, 2015] together with selective learning [Fu et al., 2026], which masks uncertain and anomalous timesteps during optimization so that the model focuses on more generalizable patterns. Empirically, the resulting strategy achieves an out-of-sample Sharpe ratio of 1.375, ranking in the top five on the JKP Common Task Framework benchmark leaderboard at the time of evaluation.

1.1 Related work/Literature review

Recent research has increasingly applied machine learning and deep learning to asset pricing, return prediction, and portfolio optimization. In empirical asset pricing, Gu et al. [2020] show that machine learning methods can generate substantial economic gains by capturing nonlinear interactions among firm characteristics, while Chen et al. [2024] use deep neural networks to estimate flexible asset pricing models with rich conditioning information and time-varying economic states. A closely related line of work studies end-to-end portfolio optimization: Zhang et al. [2020] directly optimize portfolio weights toward the Sharpe ratio, and Zhang et al. [2021] develop a general deep learning framework that bypasses separate return and covariance estimation. In parallel, recent time series forecasting studies have developed Transformer-based architectures for long-horizon prediction, such as Informer, FEDformer, PatchTST, and TimeXer [Zhou et al., 2021, 2022, Nie et al., 2023, Wang et al., 2024], while newer work explores KANs as an alternative nonlinear modeling component [Liu et al., 2025, Jiang et al., 2025]. Our paper differs from these studies in three ways. First, rather than focusing only on return forecasting, we directly map historical stock returns, Fama–French factors, and market news sentiment into portfolio weights optimized for Sharpe ratio. Second, we specifically investigate the role of the Transformer feed-forward network by replacing its neural network component with KANs, a direction that remains largely unexplored in financial time series forecasting. Third, we combine this Transformer–KANs design with selective learning to mitigate the effect of noisy and anomalous financial observations during training [Fu et al., 2025].

1.2 Notation

The notation used in this paper is summarized in Table 1.

Table 1: Notation used in this paper.

Symbol	Description
$\mathbf{X}_t$	Historical financial information available at decision date t .
$f_\theta(\cdot)$	Portfolio-allocation model parameterized by θ .
$\hat{\mathbf{w}}_{t+1}$	Portfolio-weight vector predicted for the next investment period.
$\mathbf{r}_{t+1}$	Vector of realized asset returns during period $t + 1$.
$r_{p,t+1}$	Realized portfolio return during period $t + 1$.
SR	Sharpe ratio of the model-generated portfolio returns.
C_y	Number of endogenous variables.
C_e	Number of exogenous and temporal variables.
d	Latent embedding dimension.
d_h	Dimension of each attention head.
d_{ff}	Hidden dimension of the KAN module.
$\mathbf{X}^{\text{en}}$	Endogenous input in $\mathbb{R}^{L \times C_y}$.
$\mathbf{X}^{\text{ex}}$	Exogenous input in $\mathbb{R}^{L \times C_e}$.
$\mathbf{X}$	Concatenated input in $[\mathbf{X}^{\text{en}}, \mathbf{X}^{\text{ex}}]$.
$\hat{\mathbf{Y}}$	Model output in $\mathbb{R}^{H \times C_y}$.
P	Length of each endogenous patch.
M	Number of non-overlapping patches, defined as $M = L/P$.
$p_{i,m}$	The m -th patch of the i -th endogenous variable.

Symbol	Description
$\mathbf{W}_p$	Linear projection matrix used for patch embedding.
$\mathbf{r}_m$	Sinusoidal positional embedding of the m -th patch.
$\mathbf{e}_{i,m}^{\text{en}}$	Embedded representation of the m -th patch of variable i .
$\mathbf{g}_i$	Learnable global token associated with endogenous variable i .
$\mathbf{E}_i^{\text{en}}$	Token sequence containing the patch embeddings and global token of variable i .
Attention($\cdot$)	Scaled dot-product attention.
MHCA($\cdot$)	Multi-head cross-attention.
KAN($\cdot$)	KAN-based nonlinear feed-forward mapping.
$\phi_{ji}(\cdot)$	Learnable univariate KAN edge function from input coordinate i to output coordinate j .
G	Grid size of the B-spline basis.
k	Order of the B-spline basis.
$B_{ir}^{(k)}(\cdot)$	The r -th B-spline basis function for input coordinate i .
$\Phi^{(q)}$	The q -th KAN transformation layer.
$\mathbf{Z}^{\text{out}}$	Flattened latent representation produced by the final encoder layer.
D	Dimension of the flattened encoder representation.
N	Number of assets in the portfolio.
$\mathbf{s}$	Vector of raw asset-allocation scores.
$\mathcal{N}(\cdot)$	Normalization operation converting raw scores into portfolio weights.
$\mathbf{w}$	Normalized portfolio-weight vector.
$\tilde{w}_{i,t+1}$	Raw portfolio weight assigned to asset i for period $t + 1$.
$w_{i,t+1}$	Normalized portfolio weight assigned to asset i for period $t + 1$.
$\mathcal{L}_{\text{train}}$	Negative-Sharpe-ratio training objective.
B	Batch size
$R_{i,t}$	Log return of asset i at time t .
$P_{i,t}$	Price of asset i at time t .

1.3 Organization

2 Preliminary

This study considers a portfolio-oriented forecasting problem. The goal is to learn an allocation rule that directly converts historical financial information into portfolio weights. In this sense, the forecasting output is a portfolio decision, and the quality of the forecast is judged by the realized performance of that decision in the next period.

At each decision date, KANformer takes as input a historical window containing endogenous stock-return information and exogenous market-level signals. Based on this information, the model produces the portfolio weights for the following investment period:

$$\hat{\mathbf{w}}_{t+1} = f_{\theta}(\mathbf{X}_t).$$

After the next-period returns are realized, the portfolio return is computed as

$$r_{p,t+1} = \hat{\mathbf{w}}_{t+1}^{\top} \mathbf{r}_{t+1}.$$

Thus, the model is evaluated through the return generated by its predicted weights, rather than through the pointwise accuracy of return forecasts.

This formulation is motivated by the mismatch between return prediction and portfolio decision-making. If a model first predicts future returns, a separate trading or market-timing rule is still required to translate those forecasts into portfolio weights. The final investment outcome then depends not only on the accuracy of the return forecasts, but also on the additional allocation rule, leverage constraint, and rebalancing decision. In noisy financial markets, this two-stage procedure may amplify prediction errors and produce unstable portfolio weights. To reduce this mismatch, we formulate the problem as a direct portfolio-weight learning task, where the model is optimized toward the Sharpe ratio of the realized portfolio returns:

$$\text{SR} = \frac{\mathbb{E}[r_{p,t+1}]}{\sqrt{\text{Var}(r_{p,t+1})}}.$$

Equivalently, the desired allocation rule can be viewed as the one that maximizes the Sharpe ratio induced by the model-generated portfolio returns:

$$\theta^* = \arg \max_{\theta} \frac{\mathbb{E} [f_{\theta}(\mathbf{X}_t)^{\top} \mathbf{r}_{t+1}]}{\sqrt{\text{Var}(f_{\theta}(\mathbf{X}_t)^{\top} \mathbf{r}_{t+1})}}.$$

This setting aligns the forecasting task with the economic objective of portfolio management: the model should not merely describe future returns but should produce allocations that achieve stronger compensation per unit of risk.

3 The Model and Methodology

3.1 KANformer: Structure Overview

KANformer is an encoder-only portfolio-allocation model designed for financial time series with exogenous variables. The model takes a historical input window and maps it directly into portfolio weights. Endogenous information is represented at the patch level, exogenous information is represented at the variate level, and an endogenous global token bridges the two information sources through cross-attention. In addition, the conventional Transformer feed-forward network is replaced by Kolmogorov–Arnold Networks (KANs), which provide learnable edge-wise nonlinear functions.

Given an input window of length L , the model input is divided into endogenous and exogenous components. Let

$$\mathbf{X}^{\text{en}} = [\mathbf{x}_1^{\text{en}}, \dots, \mathbf{x}_L^{\text{en}}] \in \mathbb{R}^{L \times C_y}, \quad (1)$$

denote the endogenous input. The exogenous input is denoted by

$$\mathbf{X}^{\text{ex}} = [\mathbf{x}_1^{\text{ex}}, \dots, \mathbf{x}_L^{\text{ex}}] \in \mathbb{R}^{L \times C_e}. \quad (2)$$

Equivalently, the raw input can be written as

$$\mathbf{X} = [\mathbf{X}^{\text{en}}, \mathbf{X}^{\text{ex}}] \in \mathbb{R}^{L \times (C_y + C_e)}. \quad (3)$$

The model produces a portfolio weight vector

$$\mathbf{w} = [w_1, \dots, w_{C_y}]^{\top} \in \mathbb{R}^{C_y}, \quad (4)$$

where w_i is the allocation assigned to asset i .

3.2 Input Embedding

3.2.1 Patch Embedding for Endogenous Variables

Let P denote the patch length and $M = L/P$ the number of non-overlapping patches. For the i -th endogenous variable, the historical sequence is divided into

$$\mathbf{p}_{i,m} = [y_{i,(m-1)P+1}, \dots, y_{i,mP}]^{\top} \in \mathbb{R}^P, \quad m = 1, \dots, M. \quad (5)$$

Each patch is projected into the d -dimensional latent space and added to a sinusoidal positional embedding:

$$\mathbf{e}_{i,m}^{\text{en}} = \mathbf{W}_p \mathbf{p}_{i,m} + \mathbf{r}_m, \quad \mathbf{W}_p \in \mathbb{R}^{d \times P}, \quad \mathbf{r}_m \in \mathbb{R}^d. \quad (6)$$

The sinusoidal positional embedding $\mathbf{r}_m$ is defined as

$$r_{m,2j} = \sin\left(\frac{m}{10000^{2j/d}}\right), \quad r_{m,2j+1} = \cos\left(\frac{m}{10000^{2j/d}}\right), \quad j = 0, \dots, \frac{d}{2} - 1. \quad (7)$$

A learnable global token $\mathbf{g}_i \in \mathbb{R}^d$ is appended to the patch tokens:

$$\mathbf{E}_i^{\text{en}} = [\mathbf{e}_{i,1}^{\text{en}}, \dots, \mathbf{e}_{i,M}^{\text{en}}, \mathbf{g}_i] \in \mathbb{R}^{(M+1) \times d}. \quad (8)$$

Stacking all endogenous variables gives

$$\mathbf{E}^{\text{en}} = [\mathbf{E}_1^{\text{en}}, \dots, \mathbf{E}_{C_y}^{\text{en}}] \in \mathbb{R}^{C_y \times (M+1) \times d}. \quad (9)$$

This representation allows self-attention to capture temporal dependencies among patches of each endogenous variable, while the global token summarizes the variable-level state.

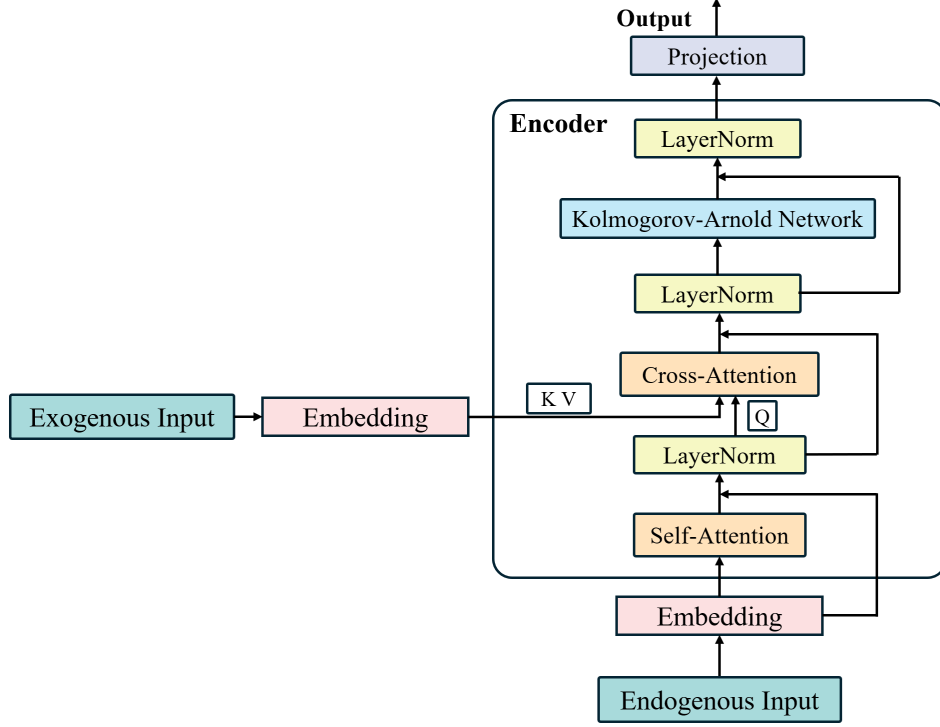


Figure 1: Architecture of the KANformer.

3.2.2 Inverted Embedding for Exogenous Variables

The exogenous variables are encoded using an inverted embedding. Instead of treating each time step as a token, this embedding treats the full historical trajectory of each variable as one token. Let

$$U = [Z, T] \in \mathbb{R}^{L \times C_e}, \quad (10)$$

where T denotes optional time features and C_e is the total number of exogenous and temporal variables. After transposing the time and variate dimensions, a linear projection maps each length- L trajectory into the latent space:

$$E^{\text{ex}} = \text{Dropout}(U^\top W_e + b_e) \in \mathbb{R}^{C_e \times d}, \quad W_e \in \mathbb{R}^{L \times d}. \quad (11)$$

Thus, each exogenous token represents the historical dynamics of one external variable over the entire input window. These tokens later serve as the key and value of cross-attention.

3.3 Encoder Layer

The model stacks N_{enc} encoder layers. The input of the ℓ -th encoder layer is

$$H^{(\ell-1)} \in \mathbb{R}^{C_y \times (M+1) \times d}. \quad (12)$$

The exogenous embedding E^{ex} is shared across all encoder layers as the source of key and value tokens for cross-attention. Each encoder layer consists of endogenous self-attention, exogenous cross-attention, and a KAN-based feed-forward mapping.

3.3.1 Endogenous Self-Attention

For each endogenous variable, the layer first applies multi-head self-attention to its patch tokens and global token:

$$\widetilde{H}^{(\ell)} = \text{LayerNorm} \left(H^{(\ell-1)} + \text{Dropout} \left(\text{Attention} \left(H^{(\ell-1)}, H^{(\ell-1)}, H^{(\ell-1)} \right) \right) \right). \quad (13)$$

The attention mechanism is the standard scaled dot-product attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}} \right) \mathbf{V}, \quad (14)$$

where d_h is the dimensionality of each attention head.

3.3.2 Exogenous Cross-Attention

Let the output of endogenous self-attention be decomposed into local patch tokens and global tokens:

$$\tilde{\mathbf{H}}^{(\ell)} = [\tilde{\mathbf{P}}^{(\ell)}, \tilde{\mathbf{G}}^{(\ell)}], \quad (15)$$

where

$$\tilde{\mathbf{P}}^{(\ell)} \in \mathbb{R}^{C_y \times M \times d}, \quad \tilde{\mathbf{G}}^{(\ell)} \in \mathbb{R}^{C_y \times d}. \quad (16)$$

Here, $\tilde{\mathbf{P}}^{(\ell)}$ contains the self-attended local patch representations, while $\tilde{\mathbf{G}}^{(\ell)}$ summarizes the endogenous sequences at the global-token level.

For cross-attention, the endogenous global tokens are used as queries, and the exogenous embedding provides the keys and values:

$$\hat{\mathbf{G}}^{(\ell)} = \text{LayerNorm} \left(\tilde{\mathbf{G}}^{(\ell)} + \text{Dropout} \left(\text{MHCA} \left(\tilde{\mathbf{G}}^{(\ell)}, \mathbf{E}^{\text{ex}}, \mathbf{E}^{\text{ex}} \right) \right) \right). \quad (17)$$

After cross-attention, the updated global tokens are concatenated with the local patch tokens:

$$\bar{\mathbf{H}}^{(\ell)} = [\tilde{\mathbf{P}}^{(\ell)}, \hat{\mathbf{G}}^{(\ell)}]. \quad (18)$$

3.4 KAN-Based Feed-Forward Mapping

A conventional Transformer encoder uses a two-layer feed-forward network of the form $\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x})$. In this model, the FFN is replaced by a KAN module:

$$\mathbf{H}^{(\ell)} = \text{LayerNorm} \left(\bar{\mathbf{H}}^{(\ell)} + \text{Dropout} \left(\text{KAN} \left(\bar{\mathbf{H}}^{(\ell)} \right) \right) \right). \quad (19)$$

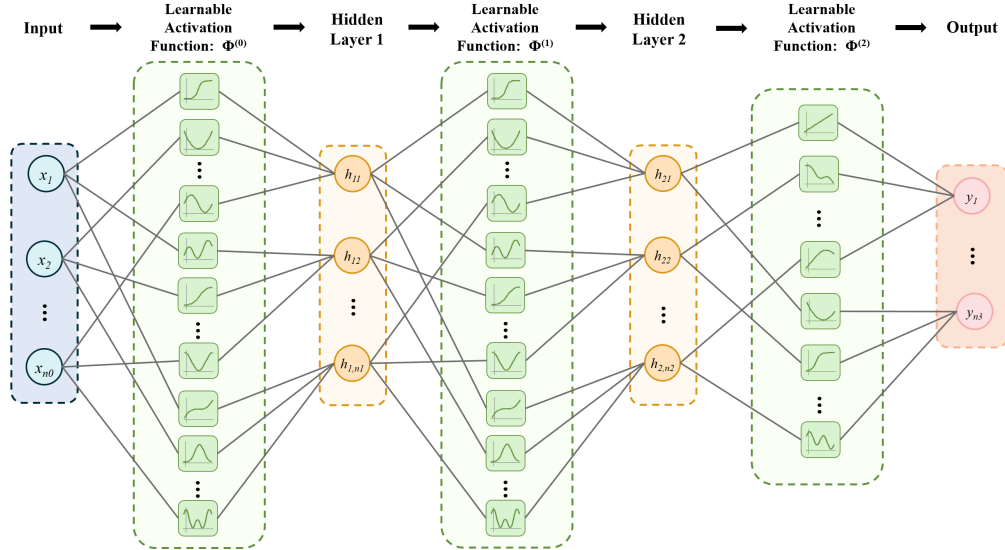


Figure 2: Illustration of KAN with learnable edge-wise activation functions. Unlike an MLP, which applies fixed activation functions at nodes, KAN learns univariate functions on edges and composes them across layers to construct high-dimensional nonlinear mappings.

For a KAN layer with input $\mathbf{u} \in \mathbb{R}^m$ and output $\mathbf{o} \in \mathbb{R}^n$, the j -th output dimension can be written as

$$o_j = \sum_{i=1}^m \phi_{ji}(u_i), \quad j = 1, \dots, n. \quad (20)$$

Each edge function $\phi_{ji}(\cdot)$ is composed of a base activation branch and a B-spline branch:

$$\phi_{ji}(u_i) = w_{ji}^{\text{base}} \text{SiLU}(u_i) + s_{ji} \sum_{r=1}^{G+k} w_{jir}^{\text{spline}} B_{ir}^{(k)}(u_i). \quad (21)$$

Here, G is the grid size, k is the spline order, $B_{ir}^{(k)}$ denotes the r -th k -order B-spline basis function for the i -th input dimension, and s_{ji} is a learnable spline scaling factor.

The two-layer KAN used in the encoder can be written as

$$\text{KAN}(\mathbf{x}) = \Phi^{(2)} \left(\Phi^{(1)} \left(\Phi^{(0)}(\mathbf{x}) \right) \right), \quad (22)$$

where $\Phi^{(0)} : \mathbb{R}^d \rightarrow \mathbb{R}^{d_{\text{ff}}}$, $\Phi^{(1)} : \mathbb{R}^{d_{\text{ff}}} \rightarrow \mathbb{R}^{d_{\text{ff}}}$, and $\Phi^{(2)} : \mathbb{R}^{d_{\text{ff}}} \rightarrow \mathbb{R}^d$. This gives the encoder a high-capacity nonlinear mapping after attention-based contextualization.

3.5 Projection Head and Portfolio Weight Normalization

After the final encoder layer, the model obtains a latent representation that contains information extracted from the endogenous input, the exogenous variables, and the KAN-enhanced nonlinear transformation. Let this final representation be denoted by

$$\mathbf{z}^{\text{out}} = \text{Flatten} \left(\mathbf{H}^{(N_{\text{enc}})} \right) \in \mathbb{R}^D, \quad (23)$$

where D is the flattened hidden dimension after the encoder. The projection head maps this representation into asset allocation scores:

$$\mathbf{s} = \mathbf{W}_o \mathbf{z}^{\text{out}} + \mathbf{b}_o, \quad \mathbf{s} \in \mathbb{R}^{C_y}. \quad (24)$$

The score vector is then transformed into a portfolio weight vector. Under the long-only fully invested constraint, this normalization is implemented by the softmax function:

$$w_i = \frac{\exp(s_i)}{\sum_{j=1}^{C_y} \exp(s_j)}, \quad i = 1, \dots, C_y. \quad (25)$$

Therefore,

$$w_i \geq 0, \quad \sum_{i=1}^{C_y} w_i = 1. \quad (26)$$

The final model output is

$$\mathbf{w} = [w_1, w_2, \dots, w_{C_y}]^\top \in \mathbb{R}^{C_y}. \quad (27)$$

4 Data and Experiments

4.1 The Data

The empirical analysis uses three classes of data.

First, stock return data are obtained from JKP Factors. The individual stock returns are drawn from the JKP Factors platform, which provides the global stock and factor database constructed by [Jensen et al., 2023]. The data system covers the U.S. stock market and is updated regularly. It includes stock returns, stock characteristics, and factor returns. This thesis uses the stock return component and performs additional screening on the raw sample. Specifically, after handling missing values, rows and columns with more than 30 percent missing observations are removed. Following this cleaning step, the average return of each stock over the sample period is computed, and the 30 stocks with the highest average returns are selected as the research universe. The JKP data used in this

thesis therefore consist not of the full market sample, but of a 30-stock subsample obtained after missing-value cleaning and return-based ranking.

Second, the Fama–French three factors are obtained from the Kenneth French Data Library [French, 2026], one of the most widely used public databases in empirical asset-pricing research. The three factors used here are the market risk premium $R_m - R_f$, the size factor SMB, and the book-to-market factor HML. $R_m - R_f$ measures the excess return on the market portfolio over the risk-free rate. SMB captures the size effect, and HML captures the value effect. These factors are canonical benchmark variables for explaining cross-sectional differences in stock returns.

Third, news-sentiment data are obtained from the Daily News Sentiment Index released by the Federal Reserve Bank of San Francisco [Federal Reserve Bank of San Francisco, 2026]. This index is constructed from economically relevant news reports through textual analysis. It uses articles from 24 major U.S. newspapers as the raw sample, retaining only articles of at least 200 words whose topic is identified as economics and whose country focus is the United States. The article-level sentiment scores are aggregated into a daily series, with recent news receiving larger weights through a geometrically decaying scheme. The resulting index is well suited as a proxy for high-frequency macroeconomic sentiment.

The study does not use a massive market-wide cross section. Instead, it selects a 30-asset subset. This choice controls experimental complexity and yields a compact portfolio weight vector that is easier to interpret economically. The scale is large enough to demonstrate the method’s effectiveness while still permitting an economically meaningful discussion.

4.2 Model Inputs and Outputs

Let $N = 30$ denote the number of selected stocks, $C_e = 4$ denote the number of exogenous variables, and $L = 192$ denote the historical input-window length. The exogenous variables consist of the Fama–French three factors and the news-sentiment factor.

For each sample indexed by h , the starting time is

$$s_h = 1 + L(h - 1), \quad h = 1, \dots, S, \quad (28)$$

where S is the number of available input windows. The return of stock i at time t is computed as the log return

$$R_{i,t} = \log \left(\frac{P_{i,t}}{P_{i,t-1}} \right). \quad (29)$$

The endogenous input of sample h is the historical return matrix

$$\mathbf{X}_h^{\text{en}} = [R_{i,t}]_{s_h \leq t \leq s_h + L - 1, 1 \leq i \leq N} \in \mathbb{R}^{L \times N}. \quad (30)$$

The exogenous input is

$$\mathbf{X}_h^{\text{ex}} \in \mathbb{R}^{L \times C_e}. \quad (31)$$

The full model input is therefore

$$\mathbf{X}_h = [\mathbf{X}_h^{\text{en}}, \mathbf{X}_h^{\text{ex}}] \in \mathbb{R}^{L \times (N + C_e)}. \quad (32)$$

Given $\mathbf{X}_h$, the model outputs a portfolio weight vector for the next investment period:

$$\mathbf{w}_h = f_\theta(\mathbf{X}_h) = [w_{1,h}, \dots, w_{N,h}]^\top \in \mathbb{R}^N. \quad (33)$$

Under the long-only fully invested setting,

$$w_{i,h} \geq 0, \quad \sum_{i=1}^N w_{i,h} = 1. \quad (34)$$

The realized next-period portfolio return is

$$r_{p,h+1} = \sum_{i=1}^N w_{i,h} R_{i,s_h+L}. \quad (35)$$

Thus, the model is trained not to predict each individual stock return first, but to learn a direct mapping from historical market information to portfolio allocations.

4.3 Loss Function

Unlike the conventional two-stage paradigm in financial forecasting, where asset returns are first predicted and then translated into portfolio allocations, this study formulates the problem directly as a portfolio-weight learning task. The optimization objective is aligned with risk-adjusted portfolio performance.

The batch setting is introduced during training. Let

$$\mathcal{B}_q = \{h_1, \dots, h_B\} \quad (36)$$

denote the q -th mini-batch, where B is the batch size. For each sample $h \in \mathcal{B}_q$, the model produces weights w_h and the realized next-period portfolio return is computed as $r_{p,h+1}$.

The sample mean of portfolio returns within the mini-batch is

$$\bar{r}_{p,\mathcal{B}_q} = \frac{1}{B} \sum_{h \in \mathcal{B}_q} r_{p,h+1}. \quad (37)$$

The corresponding batch variance is

$$\sigma_{p,\mathcal{B}_q}^2 = \frac{1}{B-1} \sum_{h \in \mathcal{B}_q} (r_{p,h+1} - \bar{r}_{p,\mathcal{B}_q})^2. \quad (38)$$

The training loss is the negative Sharpe ratio computed over the mini-batch:

$$\mathcal{L}_{\text{train}} = -\frac{\bar{r}_{p,\mathcal{B}_q}}{\sqrt{\sigma_{p,\mathcal{B}_q}^2 + \varepsilon}}, \quad (39)$$

where ε is a small numerical stabilization term. Compared with conventional regression losses such as mean squared error or mean absolute error, this objective directly optimizes the trade-off between return and risk. The mini-batch is therefore a training device used to estimate the Sharpe-oriented objective stochastically; it is not part of the model architecture itself.

4.4 Training

We employ the Adam optimizer [Kingma and Ba, 2015] for model training because its adaptive first- and second-moment estimates are well suited to stochastic optimization with noisy gradients. This property is especially useful in financial forecasting, where return-related targets are non-stationary, heavy-tailed, and affected by abrupt regime changes.

In addition, we incorporate Selective Learning during training [Fu et al., 2025]. This idea is well aligned with financial time series, whose empirical distributions often contain volatility clustering, jumps, and outlier events [Cont, 2001]. Treating all observations as equally informative may cause the model to overfit market shocks that are not representative of future regimes. By masking uncertain and anomalous timesteps, selective learning reduces the influence of non-generalizable observations and encourages the model to learn more stable temporal patterns.

Training proceeds by drawing mini-batches of input windows. For each mini-batch, the model maps every input window X_h into a portfolio weight vector w_h , computes the realized next-period portfolio return $r_{p,h+1}$, evaluates the negative Sharpe loss over the mini-batch, and updates the model parameters through backpropagation. The validation and test sets are kept separate from the training set and are used to monitor out-of-sample performance.

Figure 3 shows the training, validation, and test Loss(−Sharpe) curves. Since the loss is defined as the negative Sharpe ratio, a more negative value indicates better Sharpe-oriented performance. The training loss initially improves from -0.0896 at epoch 1 to -0.1008 at epoch 2, becomes unstable at epoch 3, and then decreases sharply to -0.1743 by epoch 5. In contrast, the validation loss does not improve in the same direction. The test loss improves slightly at epoch 2, reaching -0.0886 , but then deteriorates to -0.0705 at epoch 4 and remains locally flat between epochs 4 and 5. This local flatness suggests that the test loss has stopped making meaningful progress by the later part of training. The divergence between the increasingly negative training loss and the weaker validation/test losses after approximately epoch 3–4 indicates an overfitting pattern.

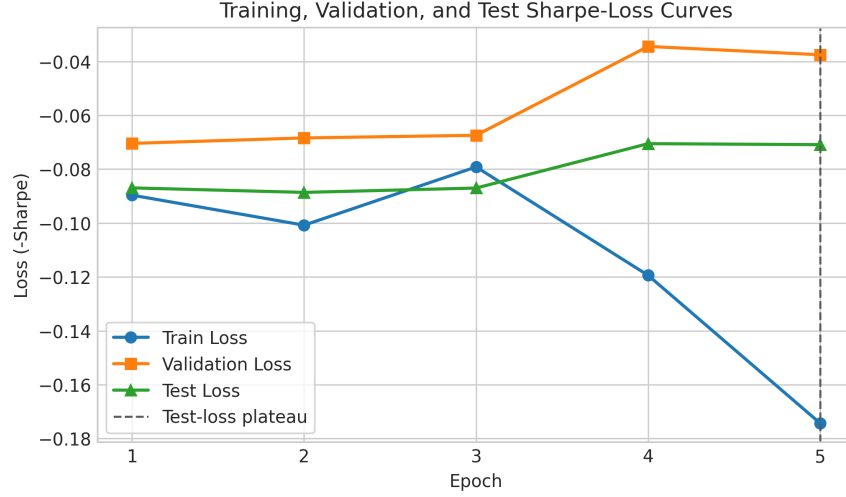


Figure 3: Training, validation, and test Loss(−Sharpe) curves. The validation and test losses become unstable after the early epochs, while the training loss continues to decrease, indicating overfitting to the training period.

Table 2: Sharpe ratio comparison with competing portfolio models.

Rank	Model	Author	Sharpe ratio
1	LightGBM + Investment Families	Darren Li	2.1854
2	MAXSER	Yingying Li	2.0261
3	ClusterSharpe Selection + MeanVar Weighting + Linear LS	Gareth Campbell	1.501
4	Bull/Bear Naïve	Kavita Kar	1.3762
5	H's model 2.0	This study	1.375
6	CSRE	Anna Siamionava	1.2591
7	Rolling Factor Selection with RankSharpe Weighting	Gareth Campbell	1.1895
8	Adaptive Ensemble with Dynamic Risk Management	Angikar Ghosal	1.0457

4.5 Results

As shown in Table 2, H's model 2.0 achieves a Sharpe ratio of 1.375, ranking No. 5 on the leaderboard. This result suggests that the proposed model obtains competitive risk-adjusted performance among related portfolio models. However, the comparison should be interpreted with caution. Our model uses both endogenous and exogenous inputs, where the endogenous component consists of return-rate data. In contrast, the other models in the table do not use exactly the same exogenous variables, and their endogenous inputs may contain a richer set of market factors. Therefore, the comparison is not fully aligned in terms of input information and prediction setting, and it should be viewed as reference evidence rather than a strict one-to-one benchmark.

References

- Tim Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of econometrics*, 31(3):307–327, 1986.
- George Box and GM Jenkins. Analysis: Forecasting and control. *San francisco*, 10, 1976.
- Luyang Chen, Markus Pelger, and Jason Zhu. Deep learning in asset pricing. *Management Science*, 70(2):714–750, 2024.

- Rama Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative finance*, 1(2):223, 2001.
- Robert F Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica: Journal of the econometric society*, pages 987–1007, 1982.
- Eugene F Fama and Kenneth R French. Common risk factors in the returns on stocks and bonds. *Journal of financial economics*, 33(1):3–56, 1993.
- Federal Reserve Bank of San Francisco. Daily news sentiment index, 2026. URL <https://www.frbsf.org/research-and-insights/data-and-indicators/daily-news-sentiment-index/>. Accessed: 2026-06-03.
- Kenneth R. French. Fama/french factors. Kenneth R. French Data Library, 2026. URL https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/Data_Library/f-f_factors.html. Accessed: 2026-06-03.
- Yisong Fu, Zezhi Shao, Chengqing Yu, Yujie Li, Zhulin An, Qi Wang, Yongjun Xu, and Fei Wang. Selective learning for deep time series forecasting. *arXiv preprint arXiv:2510.25207*, 2025.
- Yisong Fu, Zezhi Shao, Chengqing Yu, Yujie Li, Zhulin An, Qi Wang, Yongjun Xu, and Fei Wang. Selective learning for deep time series forecasting. *Advances in neural information processing systems*, 38:98084–98115, 2026.
- Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5):2223–2273, 2020.
- Theis Ingerslev Jensen, Bryan Kelly, and Lasse Heje Pedersen. Is there a replication crisis in finance? *The Journal of Finance*, 78(5):2465–2518, 2023.
- Lingyu Jiang, Yuping Wang, Yao Su, Shuo Xing, Wenjing Chen, Xin Zhang, Zhengzhong Tu, Ziming Zhang, Fangzhou Lin, Michael Zielewski, et al. Kanmixer: Can kan serve as a new modeling core for long-term time series forecasting? *arXiv preprint arXiv:2508.01575*, 2025.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary transformers: Exploring the stationarity in time series forecasting. *Advances in neural information processing systems*, 35: 9881–9893, 2022.
- Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. itransformer: Inverted transformers are effective for time series forecasting. In *International conference on learning representations*, volume 2024, pages 11116–11140, 2024.
- Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljagic, Thomas Hou, and Max Tegmark. Kan: Kolmogorov–arnold networks. In *International conference on learning representations*, volume 2025, pages 70367–70413, 2025.
- Donghao Luo and Xue Wang. DeformableTST: Transformer for time series forecasting without over-reliance on patching. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=B1Iq1EOiVU>.
- Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers, 2023. URL <https://arxiv.org/abs/2211.14730>.
- Xihao Piao, Zheng Chen, Taichi Murayama, Yasuko Matsubara, and Yasushi Sakurai. Fredformer: Frequency debiased transformer for time series forecasting. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, pages 2400–2410, 2024.
- Tabish Ali Rather, SM Joy, Nadezda Sukhorukova, and Federico Frascaoli. Kan vs lstm performance in time series forecasting. *arXiv preprint arXiv:2511.18613*, 2025.
- William F Sharpe. Mutual fund performance. *The Journal of business*, 39(1):119–138, 1966.

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- Yuxuan Wang, Haixu Wu, Jiayang Dong, Guo Qin, Haoran Zhang, Yong Liu, Yunzhong Qiu, Jianmin Wang, and Mingsheng Long. Timexer: Empowering transformers for time series forecasting with exogenous variables. *Advances in Neural Information Processing Systems*, 37:469–498, 2024.
- Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting, 2022. URL <https://arxiv.org/abs/2106.13008>.
- Chao Zhang, Zihao Zhang, Mihai Cucuringu, and Stefan Zohren. A universal end-to-end approach to portfolio optimization via deep learning. *arXiv preprint arXiv:2111.09170*, 2021.
- Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deep learning for portfolio optimization. *arXiv preprint arXiv:2005.13665*, 2020.
- Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting, 2021. URL <https://arxiv.org/abs/2012.07436>.
- Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting, 2022. URL <https://arxiv.org/abs/2201.12740>.